Basi di dati

(aiuto!!!)

Matteo Balzerani

October 2025

Contents

I Teoria **2**

1 Introduzione **2**

1.1 Moduli di un DBMS 3

1.2 Condivisione di una BD 3

1.3 Gestione della memoria secondaria 4

1.3.1 Buffer 4

1.3.2 Gestione del buffer 4

2 Possibili domande d'esame **6**

Descrizione Esame

- **Modulo di Teoria** Prova scritta che consiste in domande ed esercizi sugli argomenti trattati. Possibilità di ripetere la prova. In caso di ripetizione della prova, l'ultimo voto ottenuto sostituisce il precedente.
- **Modulo di Laboratorio** Prova scritta che consiste in esercizi sugli argomenti trattati. Il voto finale dell'insegnamento di Basi di Dati e Web sarà dato dalla media tra il voto della prova scritta di teoria e il voto delle prova scritta di laboratorio. Il voto complessivo verrà pubblicato anche sulla piattaforma MyUnivr e dopo l'accettazione verrà verbalizzato.

Part I
Teoria

1 Introduzione

Cos'è un DBMS? Sistema (prodotto software) in grado di gestire collezioni di dati che siano:

- **grandi** (di dimensioni (molto) maggiori della memoria centrale dei sistemi di calcolo utilizzati). La grandezza richiede che tale gestione sia sofisticata (non possiamo caricare tutto in memoria principale e poi riscaricare).
- **persistenti** (con un periodo di vita indipendente dalle singole esecuzioni dei programmi che le utilizzano). La persistenza richiede una gestione in memoria secondaria.
- **condivise** (utilizzate da applicazioni diverse)

garantendo affidabilità (resistenza a malfunzionamenti hardware e software) e privatezza (con una disciplina e un controllo degli accessi). Come ogni prodotto informatico, un DBMS deve essere efficiente (utilizzando al meglio le risorse di spazio e tempo del sistema) ed efficace (rendendo produttive le attività dei suoi utilizzatori).

Architettura a livelli dei DBMS I DBMS sono organizzati secondo una architettura a livelli

- Livello **LOGICO**: a cui fanno riferimento gli utenti (SQL).
- Livello **FISICO**: stabilisce l'effettiva implementazione in modo efficiente

  Esempio: le relazioni del modello relazionale dei dati vengono realizzate utilizzando varie possibili strutture fisiche

La proprietà di INDIPENDENZA dei dati permette agli utenti di ignorare le strutture fisiche – SQL fa riferimento al livello logico – Il DBMS si preoccupa di tradurre le operazioni al livello più basso, utilizzando in modo efficiente le strutture fisiche. Gli utenti vedono il modello logico (relazionale).
I dati sono in **memoria secondaria**, le strutture logiche non sarebbe efficienti in memoria secondaria: servono strutture fisiche opportune. La memoria secondaria è molto più lenta della memoria principale: serve un'interazione fra memoria principale e secondaria che limiti il più possibile gli accessi alla secondaria.

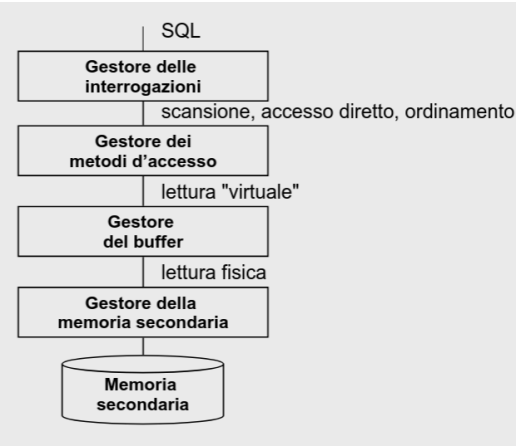


Figure 1: Architettura della gestione degli accessi e delle interrogazioni in un DBMS

1.1 Moduli di un DBMS

Domanda d'esame: ***Si descrivano i moduli che compongono il DBSM descrivendo brevemente le funzioni di ciascuno:*** risposta nelllo schema sottostante. Bisogna parlare di ciascun modulo brevemente, facendo l'esempio di una query dall'alto verso il basso e viceversa.

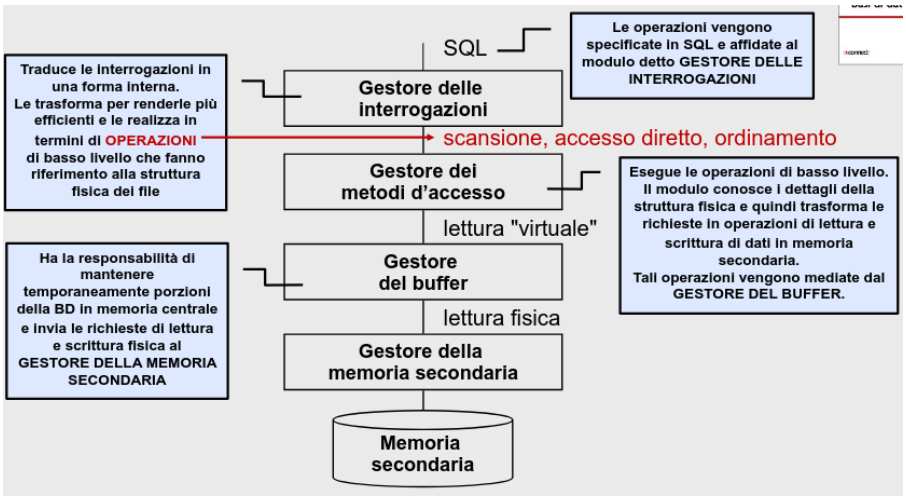


Figure 2

Gestore delle interrogazioni Prende la query logica e la traduce in una forma interna. Le trasforma per renderle più efficienti e le realizza in termini di **operazioni di basso livello** che fanno riferimento alla struttura fisica dei file.

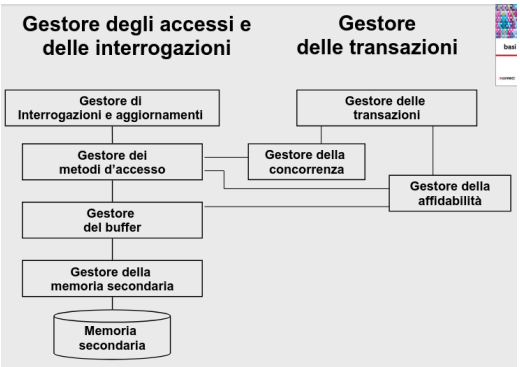
Gestore dei metodi di accesso Esegue le operazioni di basso livello. Il modulo conosce i dettagli detta struttura fisica e quindi trasforma le richieste in operazioni di lettura e scrittura di dati in memoria secondaria. Tali operazioni vengono mediate dal GESTORE DEL BUFFER.

Gestore del buffer Ha la responsabilità di manetenere temporaneamente porzioni della BD in memoria centrale e invia le richieste di lettura e scrittura fisica al **gestore della memoria secondaria.**

Transazioni le basi di dati si basano sul concetto di transazione (insieme di operazioni), che deve essere **atomica**(o tutto o niente) e **definitiva** (dopo che è andata a buon fine, deve rimanere permanente la transazione eseguita).

1.2 Condivisione di una BD

Attività diverse su dati in parte condivisi, questo richiede **meccanismi di autorizzazione** e attività multi-utente su dati condivisi: richiedono **controllo della concorrenza.** L'accesso concorrente in scrittura deve essere regolato.



Il gestore della concorrenza e il getore della affidabilità sono sottomoduli del gestore delle transazioni (DBMS sistema informativo transazionale.) *Quali sono i moduli che mi permettono di gestire le transazioni in un DBMS?* Significa gestire la concorrenza e gestire l'affidabilità.

1.3 Gestione della memoria secondaria

Il costo di un accesso a memoria secondaria è quattro o più ordini di grandezza maggiore di quello per operazioni in memoria centrale. Nelle applicazioni "I/O bound" (cioè con molti accessi a memoria secondaria e relativamente poche operazioni) il costo dipende esclusivamente dal numero di accessi a memoria secondaria. Nei DBMS l'interazione tra memoria centrale e secondaria è realizzata attraverso un buffer.

1.3.1 Buffer

- Area di memoria centrale, gestita dal DBMS (preallocata) e condivisa fra le transazioni.
- Organizzato in pagine di dimensioni pari o multiple di quelle dei blocchi di memoria secondaria. (Per semplicità ipotizzeremo che ciascuna pagina corrisponda ad un blocco)
- L'allocazione di buffer sempre più grandi, riduce il numero di accessi alla memoria secondaria
- La gestione ottimale del buffer è un aspetto fondamentale nelle BD (rispetto alle prestazioni). Il buffer è importantissimo per via della grande differenza di tempo di accesso fra memoria centrale e memoria secondaria.

1.3.2 Gestione del buffer

Scopo della gestione del buffer

- **Ridurre il numero di accessi alla memoria secondaria***GARANTENDO L'AFFIDABILITÀ.*
  - ◇ In caso di **lettura**, se la pagina è già presente nel buffer, non è necessario accedere alla memoria secondaria.
  - ◇ In caso di **scrittura**, il gestore del buffer può decidere di differire la scrittura fisica (ammesso che ciò sia compatibile con la gestione dell'affidabilità)
- Le politiche di gestione del buffer assomigliano a quelle della gestione della memoria centrale da parte dei Sistemi Operativi: obbediscono allo stesso principio, detto di località, in base al quale i dati referenziati di recente hanno una maggiore probabilità di essere referenziati nuovamente in futuro

Il buffer offre ai moduli di livello più alto una interfaccia complessa, non si limita alle richieste di lettura e scrittura. Ha bisogno delle informazioni sul prevedibile riutilizzo delle pagine (per evitarne la riapertura); Ha bisogno delle informazioni sulla necessità di effettuare immediatamente gli aggiornamenti (per evitare la perdita in caso di guasto). Per la gestione dei buffer sono necessarie le seguenti informazioni:

- Un direttorio che per ogni pagina mantiene (ad esempio) il file fisico e il numero del blocco ad essa corrispondente.
- Per ogni pagina del buffer il gestore mantiene due variabili di stato: **un contatore** che indica quanti programmi utilizzano la pagina e un **bit** che indica se la pagina è "sporca", cioè se è stata modificata

Funzioni del buffer manager Le politiche sono simili a quelle relative alla gestione della memoria da parte dei sistemi operativi.

Principio di "località dei dati": è alta la probabilità di dover riutilizzare i dati attualmente in uso.
Legge 80-20: l'80% delle operazioni utilizza sempre lo stesso 20% dei dati.

Operazioni per la gestione del buffer:

- **Fix:** Richiesta di accesso ad una pagina, richiede una lettura solo se la pagina non è nel buffer (incrementa il contatore associato alla pagina), ritorna il riferimento alla pagina nel buffer. Cerca la pagina nel buffer: se c'è, restituisce l'indirizzo; per il principio di località ciò avviene abbastanza spesso. Altrimenti, cerca una pagina libera nel buffer (contatore a zero): Se la trova, restituisce l'indirizzo; se il BIT di stato segnala che la pagina è stata modificata, essa viene aggiornata in memoria di massa. Altrimenti (se non ci sono

pagine libere), due alternative – "steal": selezione di una "vittima", pagina occupata del buffer; i dati della vittima sono scritti in memoria secondaria; viene letta la pagina di interesse dalla memoria secondaria e si restituisce l'indirizzo – "no-steal": l'operazione viene posta in attesa (che si liberino pagine nel buffer)

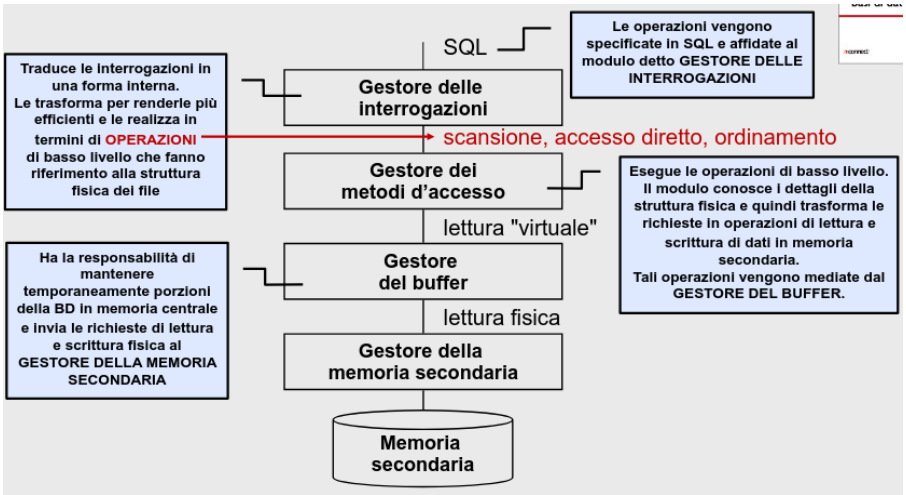
- **setDirty** Comunica al buffer manager che la pagina è stata modificata. Viene modificato il BIT di stato relativo.
- **Unfix:** indica che la transazione ha concluso l'utilizzo della pagina, decrementa il contatore associato alla pagina.
- **Force:** Trasferisce in modo sincrono una pagina in memoria secondaria **su richiesta del gestore dell'affidabilità** (non del gestore degli accessi) per garantire che alcuni dati non vengano persi. Memorizza in maniera permanente le modifiche per garantire l'affidabilità.

La scrittura in memoria secondaria può avvenire in due modi:

- in modo sincrono quando è richiesto esplicitamente con una force
- in modo asincrono (indipendente dall'applicazione) quando il buffer manager lo ritiene opportuno (o necessario). In particolare, può decidere di anticipare o posticipare scritture per coordinarle e/o sfruttare la disponibilità dei dispositivi

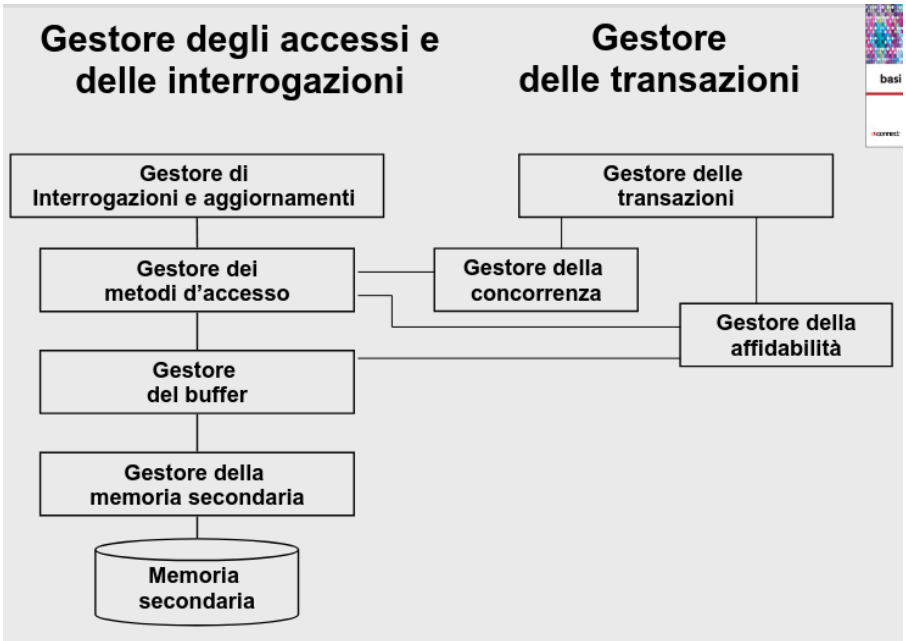
2 Possibili domande d'esame

- ***Si descrivano i moduli che compongono il DBSM descrivendo brevemente le funzioni di ciascuno:*** risposta nelllo schema sottostante.



Bisogna parlare di ciascun modulo brevemente, facendo l'esempio di una query dall'alto verso il basso e viceversa.

- ***(da approfondire) Quali sono i moduli che mi permettono di gestire le transazioni in un DBMS?*** Significa gestire la concorrenza e gestire l'affidabilità.



- ***Si descriva in che modo le tuple vengono organizzate nelle pagine:***

B + Tree F Chiavi, ogni chiave è seguita da un puntatore. La chiave K_1 ha un puntatore prima di sè P_0, quindi abbiamo $F + 1$ puntatori che si chiama **Fan Out dell'albero**. (Dato Fan Out =4, abbiamo 3 chiavi). I puntatori saranno sempre uno in più, il Fan Out dice il numero massimo di puntatori che posso avere. In questo caso il puntatore P_0 che sta prima della prima chiave punta al sottoalbero che contiene le chiavi $K < K_1$. P_F punta al sottoalbero che contiene le chiavi $K \geq K_F$ mentre P_i è il puntatore che punta al sottoalbero che contiene le chiavi $K_i \leq K < K_{i+1}$.